

Manjula H M

H M MANJULA - final_waf_paper

 Quick Submit Quick Submit Presidency University

Document Details

Submission ID

trn:oid::1:3519846581

Submission Date

Mar 28, 2026, 11:01 AM GMT+5:30

Download Date

Mar 28, 2026, 11:05 AM GMT+5:30

File Name

final_waf_paper.docx

File Size

1.4 MB

7 Pages

6,288 Words

37,463 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Transformer based end-to-end Web Application Firewall (WAF) pipeline

Dr. Manjula HM¹, Praveen R², Kushal V³, Mahesh Babu P⁴

Dept. of Computer Science and Technology, Presidency University, Bangalore, India.

(praveenrr405@gmail.com)

ABSTRACT

Web applications are increasingly becoming susceptible to various complex cyber attacks such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal, and Command Injection. Conventional signature-based Web Application Firewalls (WAFs) are largely rule and pattern-based making them ineffective against zero-day and obfuscated attacks. In response to these issues, the proposed research implements a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF), which uses deep learning-based methodologies to conduct real-time analysis of the HTTP request and classify threats based on sophisticated measures. The proposed system consists of a fine-tuned DistilBERT transformer model that is used to examine contextually enriched HTTP requests. Semantic features, special character patterns, and payload structural patterns are obtained with the aid of a custom Request Encoder before tokenization, which is needed to aid in contextual understanding. The transformed encoded input is then provided into the transformer model which contains self-attention mechanisms to identify complex relationships between the request elements. The model provides predictions regarding the probabilities of all the five types: SAFE, SQLi, XSS, PATHTRAVERSAL, and COMMANDINJECTION. A confidence-sensitive decision engine is provided to help with more trustworthy security decisions. The suggested system categorizes the threats as being of high, medium or low confidence in their regions using softmax-based confidence scores as opposed to conventional models that merely use predicted class labels. Very likely malicious requests are blocked immediately, somewhat likely malicious requests are recorded with warnings, and not very likely malicious ones are set to be analyzed manually. This multi-layered decision making approach has the advantage of eliminating false positives but offers good threat coverage. The results of testing the proposed AI-WAF on curated and unseen sets of HTTP requests indicate that the given AI-WAF has the overall accuracy of 90.62% on unseen test data and a macro F1-score of 0.9125. In nearly all kinds of attacks, the system maintains an accuracy of 100 percent and in the critical injection-based attacks, the system reduces the number of false negatives significantly. Findings indicate that transformer-based contextual modeling using confidence-sensitive decision logic can be a robust, general, and scalable method of securing web applications in the present era.

Keywords: Artificial Intelligence, Web Application Firewall (WAF), Transformer Model, DistilBERT, Deep Learning, Cybersecurity, Confidence-Aware Security.

I. INTRODUCTION

The high proliferation of web-based applications has exposed contemporary digital infrastructures to attacks significantly. Increasingly, web services are being utilized by businesses to store information, make telephone calls as well as conduct business in the cloud. However, the same growth has caused further sophisticated web application attacks such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal and Command Injection. The vulnerability of injection is always ranked among the highest risk of web security according to reports on web security all over the world. They may cause significant information leakage and losses.

The classical Web Application Firewalls (WAFs) are mainly based on rule-based systems of detection and static signature matching. Although it

is a good solution to known attack patterns, these systems find it hard to identify obfuscated payloads, polymorphic attacks and zero-day exploits. Assailants often alter the format of payload, encode malicious data or apply hybrid injection attacks to overcome standard security filters. Consequently, WAFs which rely on rules can currently generate a high false-positive or fail to identify new threats.

The new opportunities of intelligent threat detection have emerged with recent developments of Artificial Intelligence (AI) and Natural Language Processing (NLP). Transformer-based models, and especially Bidirectional Encoder Representations from Transformers (BERT) and more lightweight ones like DistilBERT, have proven to have outstanding performance in contextual relationships in textual data. Transformer models are highly appropriate in semantic threat analysis since the HTTP request messages include structured textual patterns, special characters, and contextual attack signature.

It is suggested that this study presents a Transformer-Based AI Web Application Firewall (AI-WAF) which can use DistilBERT to classify HTTP requests based on their context. In contrast to the traditional frameworks, the offered framework analyzes the enriched request representations, which reflect the relationships between the methods, paths, query parameters, headers, and body content. The model classifies requests into five categories namely, SAFE, SQLi, XSS, PATHTRAVERSAL, and COMMANDINJECTION.

Moreover, it has a confidence-sensitive decision engine so as to increase reliability. The system uses scores based on the level of prediction confidence to dynamically decide the blocking threshold, logging policies and threat levels, as opposed to binary allow/block decisions. This eliminates false alarms, yet it maintains a high rate of attack prevention.

The proposed architecture is an intelligent security architecture that brings machine learning, real-time inference, structured logging, and a monitoring dashboard together into one platform. The system shows that contextual learning through transformers can be used to a large extent to enhance the protection of web applications against the emerging cyber threats.

II. LITERATURE REVIEW

Recent developments in Transformer-based architectures have greatly contributed to the research in cybersecurity especially in vulnerability detection, malware analysis, intrusion detection, and also Web Application Firewall (WAF) orchestration.

Thapa et al. [1] examined the use of transformer-based language model in detecting software vulnerability in C/C++ source code. Their model combines preprocessing, tokenization, embedding, and fine-tuning on models BERT, DistilBERT, RoBERTa, and CodeBERT. It has been shown that the experimental results are more precise and have better recall and F1-scores, which is why the attention mechanisms prove to be more effective when identifying complex vulnerability patterns.

Mamede et al. [2] suggested an IDE transformer-based VDet (Vulnerability detecting) plug-in to detect vulnerabilities in Java programs in real time. With JavaBERT refined to multi-label classification of CWE types, the system had reached 98.9% accuracy, which shows that it is indeed possible to directly apply transformer models to software development settings.

Rudd et al. [3] examined the possibility of using Transformers to the end-to-end tasks of InfoSec malicious URLs detection and malware categorization with raw Data in the form of a sequence of bytes. Their research demonstrated that the transformer architectures are capable of taking raw security data directly without feature engineering (although, the computational efficiency is still an issue).

Darwinkel [4] used transformer-encoded embeddings on HTTP response headers to perform fingerprinting of web servers, and high macro F1-scores were obtained in classification. This paper exhibits how transformers can extract the contextual patterns in the communication based on the HTTP protocol that is applicable to web traffic surveillance system.

Alshomrani et al. [5] have given an extensive review on transformer-based malware detection system, citing its superiority to the conventional signature-based systems in dealing with zero-day and polymorphic attacks. On the same note, Latibari et al. [6] investigated security issues in transformer architectures where they addressed the issue of adversarial threats and the necessity to deploy strategies that are resistant to vulnerable devices.

The interpretable federated transformer framework proposed by De La Torre Parra et al. [7] focuses on explainable and privacy-preserving AI in security systems, offering cloud log anomaly detection. Ravi et al. [8] suggested a classification framework of encrypted SaaS traffic using hybrid transformers in CASB architectures, which is flexible to unknown services.

Rahman et al. [9] introduced an AI-enhanced conceptual framework of WAF that is meant to safeguard the critical infrastructure against zero-day exploits. Their CI-AWARE framework combines the transformer-based contextual examination with the layer of anomaly recognition to create a reference framework of AI-driven WAF frameworks.

To evaluate the attention-based models, Black et al. [10] put forward the FADE dataset, a large-scale benchmark of realistic HTTP payloads used to test firewall attacks, and enable evaluation of the models. Szabo and Bilicki [11] examined the topic of GPT-based source code vulnerability inspection by showing the increased generalizability of large language models to security analysis.

There is a review of high-performance computing strategies in scalable transformer training by Hasan and Islam [12], which focuses on distributed optimization and cyber-resilient infrastructure. Ali [13] presented a threat detection system that is an AI-based system of detecting DNS over HTTPS when using advanced transformer-based architecture. Lastly, Ferla [14] has shown how machine learning models can be integrated into cloud WAF systems to detect bots, as well as classify HTTP traffic.

Taken together, these papers define the increasing role of transformer architectures in cybersecurity applications. Nevertheless, little studies have been made on the application of lightweight transformer-based WAF systems with real-time confidence-based decision mechanisms and visualization dashboards. This gap is filled by the proposed work that develops a production-focused Transformer-based AI WAF with structured logging, confidence-based blocking, and frontend interactive monitoring.

III. DATASET EXPLAINATION

The quality and variety of the dataset used to train and evaluate a machine learning-based security system are of great importance to its performance. In this study, a dataset of HTTP requests was made to replicate a realistic web traffic that had both benign and malicious requests. The dataset is made up of structured request samples that represent various types of interaction with a web application and

attack code that is frequently seen in the real-world web security environment.

The dataset is represented by the HTTP request strings and the labels on whether the request is benign or related to a particular attack type. In every request, one has the following components: the HTTP method, request path, query parameters, and the payload data. These factors act as contextual data that can make the transformer model consider the relationship of request tokens and detect malicious patterns.

The data is classified into five groups of the typical web application threats. These are SAFE, SQL Injection(SQLI), Cross-site Scripting (XSS), Path Traversal and Command Injection attacks. SAFE requests are valid user activities on the web applications whereas the other classes have malicious payloads that will be used to manipulate the vulnerabilities of applications.

The database manipulation patterns that are normally present in SQL Injection requests include logical operator, UNION queries or embedded database commands in the URL parameters. XSS samples contain JavaScript injection codes that are designed to run malicious JavaScript behind the browser of a user. Path Traversal attacks are referred to as the attacks that strive to access restricted directories through manipulations of file paths through a series of commands like: ../. The Samples of Command Injection entail system-level commands, which are contained in the parameters, and which seek to execute unauthorized commands on the server.

The preprocessing of the dataset and its subdivision into training, validation, and testing subsets were made to guarantee balanced learning and adequate model assessment. The transformer model is trained on the training dataset and the validation set helps to tune the hyperparameters and avoid overfitting. The last test data is applied to test the performance of the model on unseen data.

A preprocessing pipeline is then used before the data is fed into the transformer model. This involves normalization of request strings, elimination of whitespace that is not necessary, encoding of special characters and formation of contextual textual representations via the custom request encoder. The encoder retrieves characteristic feature of the structure like the HTTP methods, query parameters, and special character patterns, which are frequently signs of bad intentions.

This raw text input is subsequently converted to a processed text representation which is tokenized with a transformer tokenizer that is compatible with the DistilBERT architecture. The process of tokenization transforms raw request texts to token sequences that can be read by the model attention mechanism.

The last data set is more realistic in terms of web application traffic and attack trends to allow the proposed AI-WAF system to acquire contextual association between HTTP requests and identify malicious payloads with high accuracy.

IV. PROPOSED SYSTEM AND METHODOLOGY

This section reports the methodology that will be used in designing and implementing the proposed Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF). The system is designed to identify and categorize malicious HTTP requests in real-time with the use of the deep learning method. The approach is a combination of several steps, such as the processing of HTTP requests, which involves contextual feature extraction by a custom request encoder, threat classification using a transformer, and a decision mechanism that uses confidence. Inbound trafficking of messages is intercepted and converted into enhanced readings that reflect the structural and semantic trends within the request content. Those representations are then fed through a fine-tuned DistilBERT transformer model which is able to find complex relationships between elements of the requests with self-attention mechanisms. A confidence-aware decision engine, depending on the probabilities of an attack, decides whether a request should be accepted,

rejected or to be inspected. This processing pipeline stack facilitates the proposed AI-WAF system to perform accurate threat detection, whilst ensuring low latency to provide high-performance web application protection in real time.

A. System Architecture

The proposed AI-WAF architecture would be based on a variety of interconnected elements that would process the request, extract contextual features, classify threats, and enforce security decisions. The system is a middleware security that is located between the client and the web application server. This architecture will result in all incoming HTTPs requests being captured and examined prior to reaching the secure application, which will allow preventing malicious code.

The architecture has several functional modules which include an HTTP request parser, a request encoder, a transformer-based classification model, a confidence-aware decision engine, and a logging and monitoring interface. The HTTP request parser performs the task of retrieving crucial parts of a request such as request method, URL path, query parameters and payload content. The resulting extracted elements are then transmitted to the request encoder which creates a contextual representation of the request through the synthesis of structural and semantic features which might represent malicious activity.

The request representation is coded and then run through a fine-tuned DistilBERT transformer model, which learns the relationship among the tokens with self-attention. The decision engine which is confidence-aware decides whether to allow, block or flag the request depending on the prediction probabilities of the model. Alongside, all security decisions and request metadata are logged by the system via a logging module and can be viewed via a web-based dashboard interface to perform real-time security analysis and administrative control. The general structure of the proposed AI-WAF system is shown in Fig. 1.

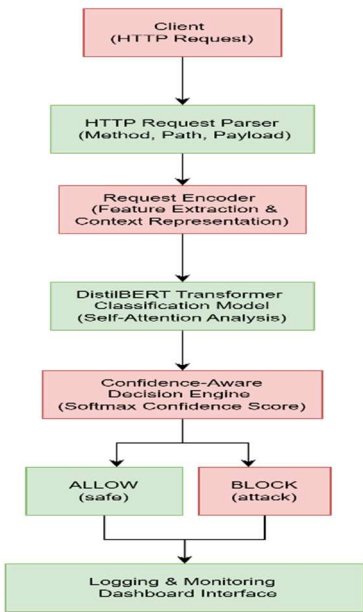


Fig. 1. Overall System Architecture

B. Data Flow of the Proposed System

The data flow of the proposed AI-WAF system shows the way that incoming HTTP requests are handled with several steps of detecting and classifying threats. The system is used as an intermediary inspection layer whereby the requests are analyzed before they are sent to the secured web application.

First, a client makes an HTTP request with a number of different elements including the request method, the URL path, query parameters, and the payload data. The AI-WAF middleware layer picks up this request and makes sure that all traffic that comes through it is analyzed and then it is sent to the application server. The captured request is then

handled through the HTTP request parser and the critical components of the request such as the HTTP method, request path, query parameters, headers and contents of the request body are extracted. The extracted request parts are sent to the request encoder following parsing to the request encoder module. A request encoder uses structural features and semantic patterns to build a contextual representation of the HTTP request, which may be a sign of malicious actions. This rich representation is then sent to the tokenizer where the input text is transformed into numerical tokens with the DistilBERT tokenizer. The tokenizer has the ability to produce input IDs and attention masks that allow the transformer model to comprehend the contextual structure of the request.

The transformation of the tokenized input is then sent to the DistilBERT-based transformer classification model. The model uses self-attention mechanisms to analyze the relationships between tokens in the request to recognize patterns of web-based attacks. The model generates scores of probability of pre-defined security classes, which are SAFE, SQL Injection (SQLI), Cross-site Scripting (XSS), Path Traversal, and Command Injection attacks.

The confidence-aware decision engine then analyses the predicted probabilities and decides what security action to take depending on the softmax confidence score. High-confidence malicious predictions are blocked on the spot, and medium-confidence cases are recorded, with a warning, and low-confidence predictions are put on the monitoring list and analyzed further. Lastly, the output of a decision and associated metadata including prediction labels, confidence scores, request information and processing latency are stored in structured logs and presented via the monitoring dashboard interface. Fig. 2 shows a general process of request processing by the AI-WAF system.

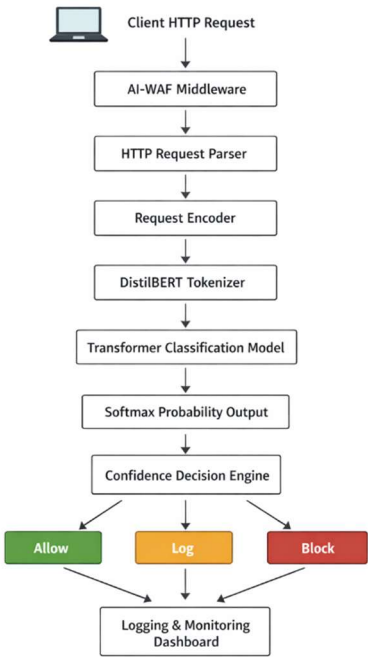


Fig. 2. Data flow diagram of AI-WAF request processing pipeline.

C. Request Encoding and Feature Extraction

A request encoding system is defined that is used as a preprocessing step to the transformer-based classification mechanism to enhance the contextual interpretation of the HTTP request. Raw HTTP requests frequently have a number of structural elements and coded payload designs that can signify ill-intentioned conducts. Hence, mere entry of raw request strings into the model might fail to give enough contextual information to make proper threat identification. In order to overcome this shortcoming, a Request Encoder module is developed to help add value to the raw request information by deriving contentful structural and semantic details.

The request encoder breaks down various attributes of an HTTP request such as request method, request URL path, query parameters and payload data. It also investigates the existence of special characters, coded symbols and suspicious patterns that are generally related to web-based attacks. An example of this is SQL injection attacks that typically contain logical operators and fragments of database queries whereas cross-site scripting attacks contain embedded script tags or JavaScript code. On the same note, path traversal attacks have directory navigation patterns, i.e. ../, and command injection attacks, i.e. shell command operators or system level instructions within parameters. Once these features have been extracted, the encoder builds a contextual representation of the HTTP request based on structural information with observed payload patterns.

Such a rich textual representation can assist the transformer model in identifying the relations among various elements of the request to detect unsafe patterns more successfully. The coded request text is then subjected to the tokenizer step which converts the text into token IDs and attention masks that are needed by the DistilBERT transformer model. The general flow of the request encoding and feature extraction employed in the suggested AI-WAF is described in Fig. 3.

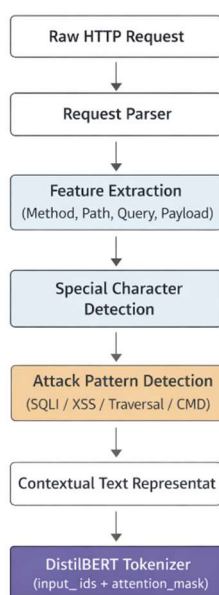


Fig. 3. Request encoding and feature extraction process for contextual representation of HTTP requests.

D. Transformer-Based Model Architecture

The suggested AI-WAF system employs an optimized DistilBERT transformer model to conduct the classification of the HTTP request. DistilBERT is a lightweight model based on the Bidirectional Encoder Representations from Transformers (BERT) that is able to maintain the performance of the original model but achieve a much lower computational cost and significantly drop the model size. This renders DistilBERT applicable to real-time security applications like web application firewall systems that have low latency and high throughput as their crucial requirements.

Within the suggested structure, the coded HTTP request obtained by the request encoder is sent to the DistilBERT tokenizer first. The tokenizer encodes the contextual request text into numerical token representations, called input IDs and outputs associated attention masks to show the valid tokens in the sequence. These tokenized inputs are then converted to dense vector representations which are used to capture the semantic meaning of the tokens.

The encodings of the embedded tokens are then subjected to multiple layers of transformer encoders with self-attention and feed-forward neural networks. The self-attention mechanism used enables the model to examine the relations between various tokens in the HTTP request and

hence identify the presence of complex contextual dependencies that can reflect malicious patterns. The operation of self-attention that is applied by the transformer models can be mathematically expressed as follows.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In which Q, K and V are the query, key and value matrices obtained based on the input embeddings, and d_k is the dimensionality of the key vectors. This mechanism makes the model to give different weights of tokens in the request sequence.

The model acquires contextual representations through these layers to generate structural and semantic features of the benign and malicious web requests. The transformer encoder generates a contextual representation which is sent to a fully connected classification layer which relates the learned features to an existing set of attack classes.

Lastly, a softmax activation function is used to create probability scores associated with each of the classes, which means the probability that the request belongs to a specific attack category. The softmax function is formulated as

$$P(y_i | x) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where z_i represents the output logit of class i , K denotes the total number of classes, and $P(y_i | x)$ represents the predicted probability of class i . The final predicted class is obtained using the **argmax function**, which selects the class with the highest probability value

$$\hat{y} = \arg \max_i P(y_i | x)$$

The proposed model categorizes the HTTP requests into five groups, which are SAFE, SQL Injection (SQLI), Cross-Site scripting (XSS), Path Traversal and Command Injection. The general transformer-based classification framework adopted by the AI-WAF system is shown in Fig. 4.

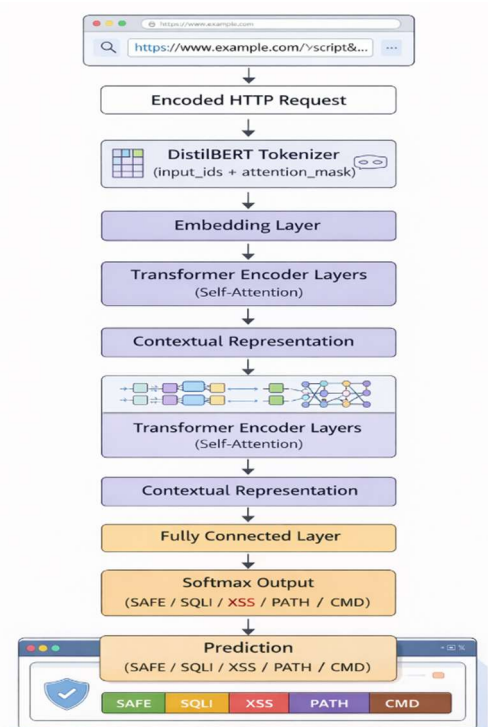


Fig. 4. DistilBERT transformer-based model architecture for HTTP request classification.

E. Confidence-Aware Decision Engine

The proposed AI-WAF uses a confidence-aware decision engine to enhance the trustworthiness of the security decisions by measuring the probability scores of the scores produced by the transformer-based classification model. Conventional machine-learning based intrusion detectors usually use only the classification as the expected output,

which can result in wrong blocking patterns in situations where the prediction made by the model is not certain. So, the suggested system applies the softmax probability scores to calculate the level of confidence of each prediction prior to executing a security measure.

Let $P(y_i | x)$ represent the predicted probability of class i for a given HTTP request x . The confidence score C is defined as the maximum predicted probability among all classes:

$$C = \max_i P(y_i | x)$$

According to the calculated confidence score, the decision engine identifies three security levels of the requests. When the score of confidence is greater than a high-confidence limit, the request will be regarded as malicious attack with high confidence and automatically blocked by the firewall. A confidence score that is in the medium-confidence category will also be described as suspicious and will be recorded to be monitored and analyzed. When the score under confidence is less than the suspicious level, the request will be viewed as safe and proceed through the system although it would be logged in the audit database.

The decision policy can thus be stated to be.

$$Decision = \begin{cases} Block, & C \geq T_{high} \\ Log, & T_{low} \leq C < T_{high} \\ Allow, & C < T_{low} \end{cases}$$

where T_{high} and T_{low} represent predefined confidence thresholds for high-risk and medium-risk requests, respectively.

This stratified decision-making model substantially lowers the possibility of false-positive results as well as offers high protection against malicious web attacks. Other related confidence-based security schemes have also been proposed in recent studies to intelligent web application firewalls and machine learning-based intrusion detection systems where probability-based decision policy is used to enhance detection reliability and system interpretability [9], [10]. The general confidence-based decision making followed in the suggested AI-WAF system is depicted in Fig. 5.

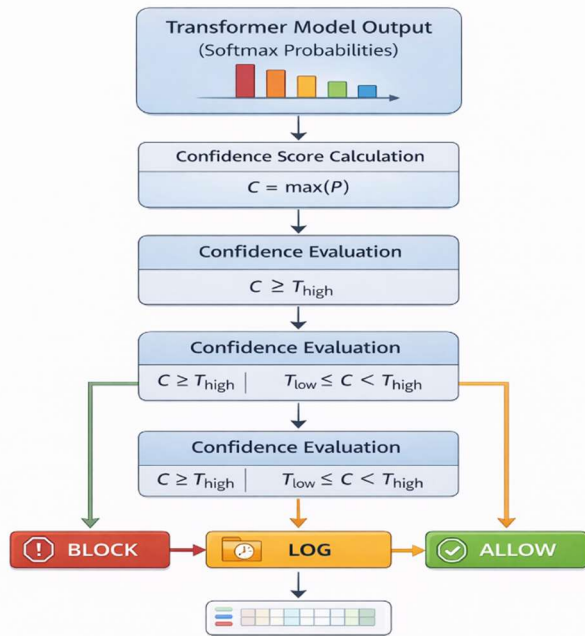


Fig. 5. Confidence-aware decision engine for HTTP request classification in the proposed AI-WAF system.

F. Logging and Monitoring System

In order to be transparent, traceable, and monitored effectively, the suggested AI-WAF will use a structured logging and monitoring mechanism which will store all the requests analyzed as well as the resulting security decisions. All requests to the system are recorded as a

structured JSON record in the log, which allows an easy storage of the security events, their retrieval, and analysis.

The logs have detailed metadata on the request and the model prediction. The common information being recorded comprises of the timestamp of the request, the method, the request path, the possible attack that the model predicts, model confidence score and the final decision made by the confidence-aware decision engine. Moreover, inference latency and contextual request parameters are also logged using the logging module to aid in performance analysis and forensic detection of possible attack patterns. The organized logging format allows full integration with the monitoring tools and security analytic offerings. This kind of elaborate logging is usually employed in contemporary web security systems to enhance the ability to respond to incidents and ease the investigation of attacks [9], [10]. The proposed AI-WAF enables security analysts to trace the actions of the system, calculate the efficiency of the models, and identify the trends of attacks by storing model predictions and decisions and request metadata.

The logged data is presented using a web-based monitoring dashboard incorporated in the AI-WAF system to give real-time visibility into the activity of the system. Live request logs, model predictions, confidence scores, and blocking decisions are shown in a live dashboard interface. This visualization will enable administrators to keep track of the activity of a system at all times and both analyze the suspicious traffic patterns and locate malicious requests swiftly. Monitoring interface is thus a valuable tool in operational security control and evaluation of the system. The general monitoring architecture, along with the dashboard interface to be applied in the proposed AI-WAF system can be shown in Fig. 6.

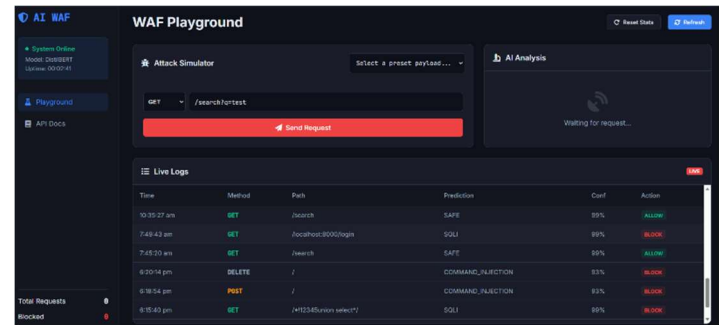


Fig .6. monitoring dashboard

V. RESULTS AND DISCUSS

After testing the proposed Transformer-Based AI Web Application Firewall (AI-WAF) performance based on a labeled HTTP request dataset that includes both attack traffic and benign traffic. Solitary web attack categories such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal and CommandInjection are present in the dataset. To avoid overfitting, the dataset was sampled into training, validation and testing subsets.

To learn contextual patterns inside the structure of an HTTP request, a DistilBERT-based classification model was trained. The trained model was incorporated into the AI-WAF system and its performance was evaluated in two ways: offline evaluation and through real-time request simulation using the web dashboard. To calculate system performance, standard machine learning metrics were used: precision, recall, F1 score and accuracy of classification. The aforementioned metrics are often used in intrusion detection and research on web security to quantify and measure the performance of detection as well and reliability (i.e. whether a system gets accurate results for predictions or not) [9], [10]. Proven by experimental results, the transformer-based model shows potential to accurately classify malicious web requests with a low latency that fits real time firewall deployment.

A. Model Performance Evaluation

Table.1. Classification Performance of the AI-WAF Model

Attack Type	Precision	Recall	F1-Score	Support
SAFE	1.00	0.88	0.94	26
SQL Injection	1.00	0.88	0.94	26
XSS	1.00	0.88	0.94	25
Path Traversal	1.00	0.88	0.94	25
Command Injection	0.68	1.00	0.81	26
Overall Accuracy			0.91	128

The result shows that the proposed AI-WAF reaches an overall classification accuracy of about 90 to 91% on its testing dataset. Moreover, high precision values for almost all of the attack classes imply that the model generates very few false positives when identifying malicious requests. Because command payload patterns used to exploit vulnerabilities share natural similarities with benign requests, some false detections might occur, which led us to a slightly lower accuracy. But the transformer model performs really well because of its self-attention, which allows the system to represent relationships between different elements in HTTP requests. Having this context enables the model to recognize complex attack patterns that are hard to detect using conventional rule-based WAF systems.

B. Confusion Matrix Analysis

The proposed AI-WAF model was further examined in terms of their classification behavior through a confusion matrix as shown below in Fig. 7. The confusion matrix will give a clear picture of the association of the predicted attack categories with the actual labels of the test data.

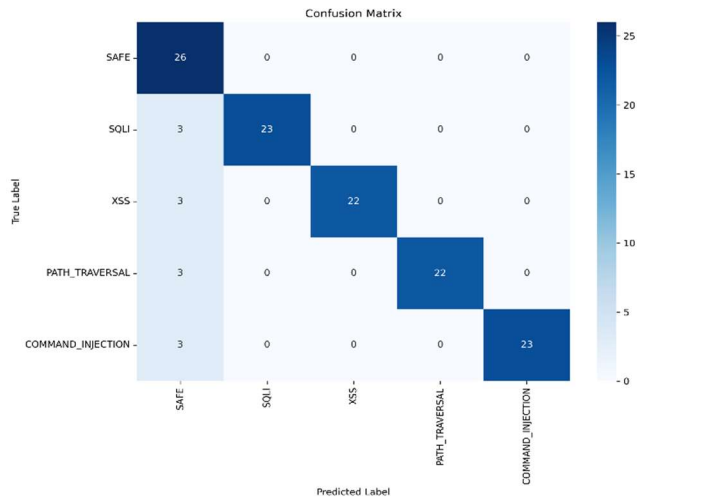


Fig. 7. Confusion Matrix

The confusion matrix can be interpreted to mean that the elements of the diagonal show those which have been classified correctly and the elements of the off-diagonal show those which have been classified incorrectly. The findings indicate that most of the HTTP requests were properly categorized in their attack classes as required. The model was able to distinguish all SAFE requests, 26 of them were rightly identified, and no false identifications. In the SQL Injection class, 23 samples were aptly identified, and 3 samples were wrongly identified as SAFE. The same case was seen with Cross-site Scripting (XSS) and Path Traversal attacks with 22 samples being characterized accurately in each category and few samples being characterized inaccurately as benign requests. Under the Command Injection type, 23 samples were rightly distinguished and 3 samples classified as SAFE.

These findings suggest that the suggested transformer-based framework has a high success rate in the classification of malicious traffic and regular traffic. The majority of misclassifications come about because some of the malicious codes look like innocent requests structurally or do not have any strong characteristics of attacks. However, the general number of correctly calculated forecasts on the diagonal of the confusion table evidences the high learning ability of the transformer-based model.

Recent studies on AI-based intrusion detection and web attack classification systems have also found the applicability of contextual feature learning with the help of transformer architectures to cybersecurity tasks [3], [9], [10]. The confusion matrix hence validates the effect that the proposed AI-WAF model is reliable in detecting various categories of web attacks and has a high classification rate.

C. Real-Time WAF System Evaluation

Besides the offline evaluation of the performance, the trained transformer-based classification model was incorporated into the proposed AI-WAF middleware so as to test its performance in a real-time setting. The system was configured as an inline request inspection layer that sits in between the HTTP requests before reaching the web application server. In order to imitate the real attack situations, several HTTP requests with good and bad payloads were created using the attack simulator interface in the monitoring dashboard.

In the assessment process, the AI-WAF system was able to identify and screen various forms of web based attacks. The system identified and blocked SQL injection attacks that had logical operators, and database query patterns. On the same note, the cross-site scripting payloads that contained embedded JavaScript execution patterns were correctly identified. The system was also successful in detecting path traversal attacks which sought to access restricted files within the system, and command injection payloads which had shell command execution patterns. Contextual feature extraction and transformer-based classification was also used on every incoming request to classify the likelihood that this request was associated with each of the defined attack categories. The model produced prediction probabilities based on the softmax activation function, and the confidence-aware decision engine was used to determine the final security decision based on the probability.

According to the calculated confidence score, the AI-WAF system automatically imposed the security policies in blocking malicious requests with high confidence and allowing legitimate requests to traverse the system. The request with a moderate level of doubt was registered on the log to be analyzed and observed. This live assessment shows that the proposed AI-WAF system can be advantageously used in real-life web security context and still retain high detection rates of web-based attacks, unlike the traditional rule-based web application firewall systems which frequently fail to detect the presence of obfuscated or previously unknown attack patterns [3], [9].

D. System Performance and Latency

Since web application firewalls operate in real-time environments, processing latency is an important performance factor. The inference latency of the transformer model was measured during request inspection.

Table .2. Average Inference Latency of the AI-WAF System

Component	Average Latency
Request Parsing	5–10 ms
Feature Encoding	10–20 ms
Transformer Model Inference	80–120 ms
Decision Engine	2–5 ms
Total Processing Time	90–150 ms

The outcomes show that the suggested AI-WAF produces an aggregate characterization precision of around 90-91% for which, as seen in the Test information set. Generally high precision values for most attack classes mean that the model is able to detect malicious requests with very few false positives. Command injection attacks are characterized with a slight

lower precision because some command payload patterns resemble benign requests, which can lead to false triggering. The strong performance of the transformer model can be helpfully to its self-attention mechanism, which enables the system to understand relationships between different elements within HTTP requests. This contextual understanding allows the model to detect complex attack patterns that are difficult to identify using traditional rule-based WAF systems.

E. Discussion

The experimental findings prove that the use of transformer-based models can considerably enhance web attack detection systems in comparison with the conventional signature-based WAF systems. Self-attention mechanisms provide the contextual interpretation necessary to facilitate the model to identify complex attack payloads that would not be recognized by the more traditional rule-based detection techniques. More so, a confidence-aware decision engine is integrated to minimize false positives by considering the predictive certainty and executing blocking decisions. Real-time visibility of the identified threats can also be observed on the logging and monitoring dashboard, thus the system can be used in both academic development and applied security systems.

The results are in line with the recent study on AI-driven security mechanisms that highlight the suitability of transformer-related architecture in cybersecurity operations like intrusion detection and web attack classification [3], [9], [10].

VI. CONCLUSION

The current paper described a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF) that was focused on identifying and rejecting the new web application attacks with the help of deep learning algorithms. The system suggested combines a DistilBERT transformer model, the encoding of contextual requests, and a confidence-sensitive decision engine to process the HTTP requests and categorize them into various categories of attack. The proposed approach is based on the notion that, unlike the traditional rule-based web application firewalls, it uses a contextual understanding of language to detect more sophisticated patterns of attack that are embedded in web requests. In this paper, a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF) has been described, which is capable of identifying and stopping the contemporary web application attacks with the application of deep learning technologies. The proposed system evaluates the HTTP requests and splits them into multiple types of attacks, based on a combination of a DistilBERT transformer model, encoding of request context, and confidence-sensitive decision engine. The proposed system involves language contextual understanding to identify complex attack patterns contained in a web request unlike the traditional rule-based web application firewalls. The assessment of the system was done using a labeled data set of 5 categories of benign and malicious HTTP requests which include; SAFE, SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection. Experimental findings indicate that transformer-based model has high classification accuracy of about 90-91% on the test data. The confusion matrix analysis further confirmed the effectiveness of transformer-based contextual learning in detecting web attacks as most samples of the attack were attributed the correct class. Also, through evaluating the probability of predictions ahead of blocking actions, evaluation of a confidence-aware decision engine, enables the system to make more reliable security decisions. This approach will have solid protection against malicious traffic and help to reduce false positives. Also, the proposed AI-WAF was deployed successfully in a real-time environment in which the middleware architecture was applied to dynamically scrutinize and categorize the HTTP requests. In the results, transformer-based models can significantly enhance web security by discovering complex attack patterns previously unknown to the conventional signature-based firewalls. Altogether, the proposed AI-WAF framework demonstrates that it can be possible to reuse transformer-based machine learning models in web security systems to provide flexible, smart, and real-time defense against modern web applications threats.

REFERENCES

Submission ID trn:oid::1:3519846581

- [1] N. Thapa, S. S. R. Depuru, and K. Roy, "Transformer-based language models for software vulnerability detection," in Proc. IEEE/ACM Int. Conf. Softw. Eng., 2022, pp. 1–12.
- [2] R. Mamede, R. Pinconschi, and F. Abreu, "A transformer-based IDE plugin for vulnerability detection," in Proc. IEEE Int. Conf. Softw. Maint. Evol., 2022, pp. 1–10.
- [3] E. M. Rudd, M. S. Rahman, and P. Tully, "Transformers for end-to-end InfoSec tasks: A feasibility study," in Proc. IEEE Secur. Privacy Workshops, 2022, pp. 1–10.
- [4] J. Darwinkel, "Fingerprinting web servers through transformer-encoded HTTP response headers," Comput. Secur., vol. 134, pp. 103–118, 2024.
- [5] M. Alshomrani, A. Alghamdi, and F. Alsubaei, "Survey of transformer-based malicious software detection systems," IEEE Access, vol. 12, pp. 45678–45702, 2024.
- [6] S. Latibari, A. Rezaei, and M. K. Rafsanjani, "Transformers: A security perspective," IEEE Access, vol. 12, pp. 78901–78925, 2024.
- [7] J. De La Torre Parra, A. Pastor, and P. García-Teodoro, "Interpretable federated transformer log learning for cloud threat forensics," IEEE Trans. Inf. Forensics Secur., vol. 17, pp. 3210–3223, 2022.
- [8] K. Ravi, S. Menon, and A. Krishnan, "CASB security analytics for encrypted SaaS traffic: A hybrid transformer-based classification framework in enterprise cloud ecosystems," IEEE Access, vol. 13, pp. 11234–11249, 2025.
- [9] M. Rahman, T. Ahmed, and L. Johnson, "AI-enhanced web application firewalls for protecting United States critical infrastructure against zero-day exploits," IEEE Access, vol. 14, pp. 55678–55705, 2026.
- [10] R. Black, D. Smith, and J. Carter, "Descriptor: Firewall attack detections and extractions (FADE)," in Proc. USENIX Secur. Symp., 2025, pp. 1–16.
- [11] A. Szabó and V. Bilicki, "A new approach to web application security: Utilizing GPT language models for source code inspection," IEEE Access, vol. 11, pp. 98765–98780, 2023.
- [12] M. Hasan and M. Islam, "High-performance computing architectures for training large-scale transformer models in cyber-resilient applications," IEEE Access, vol. 10, pp. 65432–65455, 2022.
- [13] A. Ali, "Next-generation AI for advanced threat detection and security enhancement in DNS over HTTPS," IEEE Access, vol. 13, pp. 33445–33463, 2025.
- [14] M. Ferla, "Enhancing cloud-based web application firewalls with machine learning models for bot detection and HTTP traffic classification," M.S. thesis, Dept. Comput. Sci., Univ. Padova, Padova, Italy, 2024.